
Artificial Intelligence

BS (CS) Spring 2026

Lab 12 Manual



Learning Objectives:

1. Supervised Learning

Lab Manual

Supervised Learning

Supervised learning is a type of machine learning where a **model is trained using labeled data**. In labeled data, each input is paired with a known output, allowing the model to learn patterns and relationships between them. This structured approach enables the model to make accurate predictions for new, unseen data.

As an example consider supervised learning as similar as guiding a student. The labeled data acts like a tutor, providing correct answers so the model (the student) can learn over time. Through repeated exposure to this labeled data, the model gradually becomes better at predicting the correct outcome on its own.

The **main goal** of supervised learning is to generalize from the labeled data so that it can make reliable predictions or decisions for unknown data in the future.

How Does Supervised Learning Work?

The process of supervised learning involves several key steps to train a model effectively:

1. **Data Preparation** – The first step is preparing the data by cleaning and pre-processing it. This includes removing any irrelevant information, handling missing values, and organizing the data into labeled pairs, where each input has a corresponding output.
2. **Training the Model** – The model is then trained using this labeled data. During training, the model analyzes the patterns in the input data and learns to map these inputs to their correct outputs. This involves adjusting the model's parameters to improve accuracy.
3. **Model Evaluation** – Once trained, the model is tested on unseen data (data it hasn't encountered before) to evaluate its performance. This helps determine how well the model can generalize beyond the training data.
4. **Making Predictions** – Finally, after the model has been trained and tested, it can be used to make predictions on new data. The better it learned during training, the more accurate its predictions will be.

Supervised Learning Algorithms

Supervised learning includes a variety of algorithms, each suited for different types of tasks. Here are some common supervised learning algorithms with a brief description:

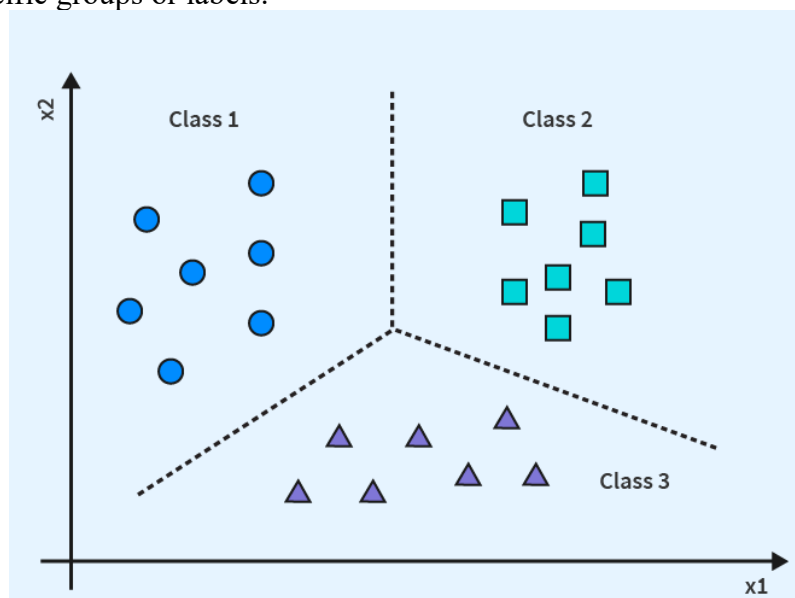
1. **Neural Networks** – Inspired by the human brain, neural networks are complex models that learn patterns in data, making them suitable for tasks like image and speech recognition.
2. **Naive Bayes** – This algorithm is based on probability and is used for classification tasks. It assumes that all features are independent, making it fast and effective for tasks like spam detection.
3. **Linear Regression** – A simple algorithm that predicts continuous values by finding a linear relationship between input features and the target variable. It's commonly used in predicting prices, like housing costs.

4. **Logistic Regression** – Despite its name, logistic regression is used for classification, not regression. It predicts binary outcomes (e.g., yes/no) and is widely used for problems like email spam detection.
5. **Support Vector Machines (SVM)** – SVMs create a hyperplane that separates data into different classes. They are effective in tasks where the data is clearly separated, such as image classification.
6. **K-Nearest Neighbors (KNN)** – KNN classifies data based on the majority label among its nearest neighbors, making it a simple but effective choice for tasks like recommendation systems.
7. **Random Forest** – This algorithm combines multiple decision trees to improve accuracy and robustness. It's effective for both classification and regression tasks and is commonly used in finance and healthcare.

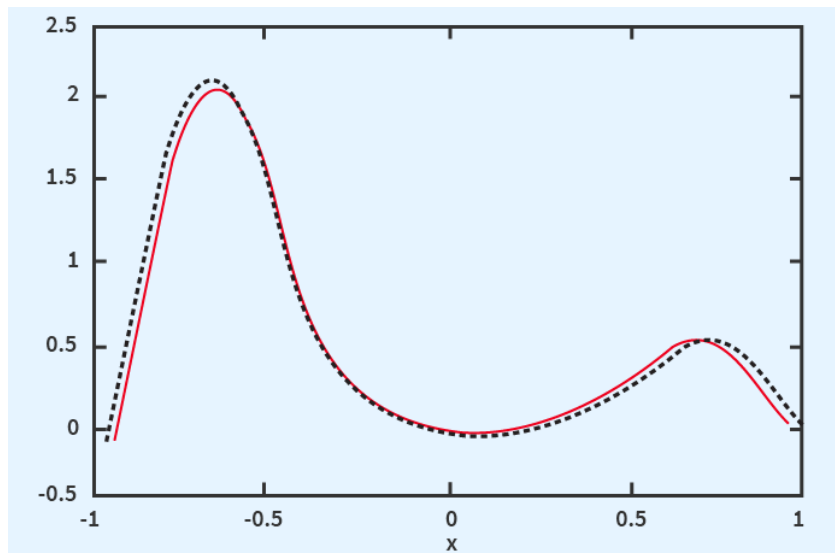
Types of Supervised Learning

Supervised learning generally falls into two main types, each designed to handle different kinds of problems:

1. **Classification** – In classification, the model predicts discrete categories or classes. For example, an email can be classified as “spam” or “not spam,” or an image can be categorized as “cat” or “dog.” Classification is useful for tasks where outcomes fall into specific groups or labels.



1. **Regression** – In regression, the model predicts continuous values. For instance, it might predict the price of a house based on features like location, size, and age. Regression is used when the output is a range of values, rather than distinct categories.



These two types help tackle a wide range of problems, from predicting sales figures to identifying objects in images, making supervised learning highly versatile across different fields.

Algorithm	Regression, Classification	Purpose	Method	Use Cases
Linear Regression	Regression	Predict continuous output values	Linear equation minimizing sum of squares of residuals	Predicting continuous values
Logistic Regression	Classification	Predict binary output variable	Logistic function transforming linear relationship	Binary classification tasks
Decision Trees	Both	Model decisions and outcomes	Tree-like structure with decisions and outcomes	Classification and Regression tasks
Random Forests	Both	Improve classification and regression accuracy	Combining multiple decision trees	Reducing overfitting, improving prediction accuracy
SVM	Both	Create hyperplane for classification or predict continuous values	Maximizing margin between classes or predicting continuous values	Classification and Regression tasks
KNN	Both	Predict class or value based on k closest neighbors	Finding k closest neighbors and predicting based on majority or average	Classification and Regression tasks, sensitive to noisy data
Gradient Boosting	Both	Combine weak learners to create strong model	Iteratively correcting errors with new models	Classification and Regression tasks to improve

				prediction accuracy
Naive Bayes	Classification	Predict class based on feature independence assumption	Bayes' theorem with feature independence assumption	Text classification, spam filtering, sentiment analysis, medical

Basic Structure

The main steps performed on any dataset generally include the following:

1. Data Collection.

2. Load and Transform Dataset

```
dataset=pd.read_csv("Live.csv")
```

3. Data Exploration and Analysis:

Understanding of relevant dataset.

1. Exploratory Data Analysis (EDA):

Understand the dataset by summarizing its main characteristics, often using visual methods.

View Basic Data:

```
dataset.head()

# Last rows
dataset.tail()

# Structure
str(dataset)

#Check shape of the dataset
dataset.shape

#Check Column Names
df.columns

#Summary
dataset.info()

#Check Duplicate Records
df.duplicated().sum()

#Checking the missing values in the dataset
dataset.isnull().sum()
```

```
#Drop redundant Columns
dataset.drop(['Column1', 'Column2', 'Column3', 'Column4'], axis=1, inplace=True)
```

```
dataset['status_type'].unique()
```

You can also perform EDA with the help of some visualizations e.g.,

```
# Plotting histograms for each attribute
dataset.hist(bins=30, figsize=(20, 15), color='b', edgecolor='black')
plt.suptitle('Histograms of All Attributes', fontsize=20)
plt.show()
```

2. Statistical Analysis:

Calculate descriptive statistics (mean, median, mode, standard deviation) to understand the data distribution.

2.1. Statistical Summary

```
#Statistical summary of numerical variables
dataset.describe()
```

2.2. Skewness and kurtosis Check

- Skewness and kurtosis are important because they describe the **asymmetry and tail behavior of data**, helping to detect outliers and check whether data meets assumptions for statistical analysis and machine learning models.

```
#Tells direction of imbalance
print("Skewness:\n", dataset.skew(numeric_only=True))
#Tells how extreme the data can get
print("\nKurtosis:\n", dataset.kurt(numeric_only=True))
```

2.3. Distribution of Engagement Features

Distribution of engagement features help us to understand user behavior patterns, detect anomalies, and ensure the data is suitable for accurate modeling and analysis.

```
#Distribution of Engagement Features
eng_cols = [
    "num_reactions", "num_comments", "num_shares",
    "num_likes", "num_loves", "num_wows",
    "num_hahas", "num_sads", "num_angrys"
]

for col in eng_cols:
    plt.figure(figsize=(5,3))
    sns.histplot(dataset[col], kde=True)
    plt.title(col)
    plt.show()
```

2.4. Correlation Analysis

Correlation analysis is used to measure the strength and direction of relationships between variables to identify which features move together and help in feature selection for modeling.

```
#Correlation Analysis
plt.figure(figsize=(10,6))
sns.heatmap(dataset.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

- We want **high correlation between independent features and the dependent variable** (useful predictors)
- We want **low correlation between independent features themselves** (to avoid redundancy / multicollinearity)

2.5. Outlier Detection

Outlier detection is the process of identifying data points that are significantly different from most of the data, as they can distort analysis and reduce model accuracy.

```
def detect_outliers(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    return dataset[(dataset[col] < lower) | (dataset[col] > upper)]

out_reactions = detect_outliers(dataset, "num_reactions")
out_comments = detect_outliers(dataset, "num_comments")

print("Outliers reactions:", len(out_reactions))
print("Outliers comments:", len(out_comments))
```

2.6. Z Score

Z-score measures how many standard deviations a data point is from the mean, and it is used to detect outliers and standardize data for comparison.

```
#Z Score
from scipy.stats import zscore

dataset["zscore"] = zscore(dataset["num_reactions"])
outliers = dataset[dataset["zscore"].abs() > 3]
print(outliers)
```

2.7. Zero Ratio Aanalysis

Zero ratio analysis measures the proportion of zero values in a dataset, and it is used to identify sparse features that may carry little information or affect model performance.

```
#Zero Ratio Analysis
zero_ratio = (dataset[eng_cols] == 0).sum() / len(dataset)
print(zero_ratio)
```

2.8. Group Analysis

Group analysis examines data by dividing it into meaningful categories to compare patterns, trends, and differences across groups for better insights and decision-making.

#Group Analysis

```
dataset.groupby("status_type")["num_reactions"].mean()  
dataset.groupby("status_type")["num_comments"].mean()
```

2.9. Hypothesis Testing

Hypothesis testing is a statistical method used to determine whether there is enough evidence in data to support or reject a specific claim or assumption.

Hypothesis Testing (t-test)

```
from scipy.stats import ttest_ind
```

```
group1 = dataset[dataset["status_type"] == "video"]["num_reactions"]  
group2 = dataset[dataset["status_type"] == "photo"]["num_reactions"]
```

```
t_stat, p_value = ttest_ind(group1, group2)
```

```
print("T-stat:", t_stat)  
print("P-value:", p_value)
```

```
from scipy.stats import chi2_contingency
```

```
table = pd.crosstab(dataset["status_type"], dataset["num_reactions"])
```

```
chi2, p, dof, expected = chi2_contingency(table)
```

```
print("Chi-square:", chi2)  
print("P-value:", p)
```

2.10. Variance Check

Variance check measures how much data values spread around the mean, and it is used to understand data variability and detect instability or inconsistency in features.

#Variance Check

```
dataset.var(numeric_only=True)
```

2.11. Features Insight

Feature insight is the understanding of how each feature behaves and contributes to the data, and it is used to guide feature selection, preprocessing, and model improvement.

#Feature Insight

```
dataset.nunique()  
dataset.describe(include="all")
```

4. Data Cleaning:

- Handling Missing Values.


```
# 1. Drop rows with missing values
df_drop = dataset.dropna()
# 2. Fill missing values (numeric columns with mean)
dataset['column_name'] = dataset['column_name'].fillna(dataset['column_name'].mean())
# 3. Fill categorical values with mode
dataset['category_column'] = dataset['category_column'].fillna(dataset['category_column'].mode()[0])
# 4. Fill all missing values (optional)
dataset.fillna(0, inplace=True)
```

- Handling Outliers.

- **Remove outliers (IQR method):** delete values outside $1.5 \times \text{IQR}$ range
- **Z-score method:** remove data points with $|Z| > 3$

```
from scipy import stats
```

```
import numpy as np
```

```
z = np.abs(stats.zscore(df['column_name']))
```

```
df = df[z < 3]
```

- **Capping (Winsorization):** replace extreme values with upper/lower limits

```
upper = df['column_name'].quantile(0.95)
```

```
lower = df['column_name'].quantile(0.05)
```

```
df['column_name'] = np.where(df['column_name'] > upper, upper,
                             np.where(df['column_name'] < lower, lower, df['column_name']))
```

- **Transformation:** use log or square root to reduce impact of extremes

```
import numpy as np
```

```
df['column_name'] = np.log1p(df['column_name'])
```

- **Imputation:** replace outliers with mean/median in some cases

- Correcting Data Types.

5. Data Preprocessing:

- **Normalization/Standardization.**

Minmax Scaler

It scales numeric values into a fixed range (usually 0 to 1) without changing categories.

```
cols = X.columns
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
ms = MinMaxScaler()
```

```
X = ms.fit_transform(X)
```

```
X = pd.DataFrame(X, columns=[cols])
```

```
X.head()
```

- **Encoding Categorical Variables.**

Label Encoder

It converts categories into numbers (e.g., Red=0, Blue=1, Green=2)

```
#Converting categorical variables into integer
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X['status_type'] = le.fit_transform(X['status_type'])
# status_type = {'photo': 0, 'video': 1}
y = le.transform(y)
X.info()
```

One-Hot Encoder

It creates separate binary columns for each category (e.g., Red→[1,0,0])

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd
encoder = OneHotEncoder(drop='first', sparse=False)
encoded = encoder.fit_transform(dataset[['column_name']])
encoded_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_out())
dataset = pd.concat([dataset, encoded_df], axis=1).drop('column_name', axis=1)
```

- Feature Selection/Extraction.

```
# Declare feature vector and target variable
```

```
X = dataset.drop('status_type', axis=1)
```

```
y = dataset['status_type']
```

- Data Splitting.

```
from sklearn.model_selection import train_test_split
# X = features, y = target variable
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(X_train.shape, X_test.shape)
```

6. Model Selection:

- Choosing the Right Model.
- Initialize the chosen models with parameters.

7. Model Training:

- Fit the model to the training data.
- Hyperparameter Tuning.

8. Model Evaluation:

- **Validation:** Evaluate the model on the validation set to tune hyperparameters and prevent overfitting.
- **Testing:** Assess the model performance on the test set using metrics like accuracy, precision, recall, F1-score, RMSE, etc.

9. Model Deployment:

- Integration.
- Monitoring and Maintenance.

10. Model Interpretation:

1. Linear Regression

Simple linear regression is a statistical method used to model the relationship between a **dependent variable (Y)** and an **independent variable (X)** by fitting a straight line through the data points. The aim is to predict the value of the dependent variable based on the independent variable using the **least squares method**.

The regression equation is represented as:

$$Y = b_0 + b_1X + \varepsilon$$

Here, **b₀** is the intercept, **b₁** is the slope of the line, and **ε** represents the error term. The slope, **b₁**, indicates how much the dependent variable changes for a unit change in the independent variable.

The **least squares method** minimizes the sum of squared differences between the observed values and predicted values to find the best-fitting line. This approach ensures the most accurate linear relationship between X and Y. Simple linear regression assumes that the relationship between the variables is linear, making it easy to interpret and apply to various datasets.

#Train the Linear Regression Model

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

Making Predictions on the Test Data:

```
y_pred = model.predict(X_test)
```

2. Logistic Regression

Logistic regression is a **supervised learning algorithm** used to estimate the probability that a given instance belongs to a particular class. It works by modeling the relationship between a categorical dependent variable and one or more independent variables. Logistic regression is particularly useful for binary outcomes, such as classifying whether an email is spam or not.

Unlike linear regression, which predicts continuous numerical values, logistic regression outputs probabilities between **0 and 1**. It achieves this by passing the linear combination of input features through a **sigmoid function**, ensuring that predictions remain within a valid probability range.

The logistic regression equation is:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1X_1 + \dots + \beta_nX_n$$

where p is the predicted probability, β_0 is the intercept, and $\beta_1, \dots, \beta_n$ are the coefficients.

```

# Model training
model = LogisticRegression()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

3. Decision Tree

A **decision tree** is a supervised learning algorithm used for both **classification** and **regression** problems. It is represented as a tree structure where each **internal node** represents a test on an attribute, each **branch** represents the outcome of the test, and each **leaf node** represents a class label or a predicted value. The goal of a decision tree is to split the dataset into subsets based on the value of an attribute, repeating this process until each subset contains only instances that belong to a single class or have similar values.

```

# Initialize the decision tree classifier
model = DecisionTreeClassifier(criterion='gini', max_depth=5)
# Train the model
model.fit(X_train, y_train)
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Make predictions
y_pred = model.predict(X_test)
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Confusion matrix and classification report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

4. K Nearest Neighbour

K-Nearest Neighbor (KNN) is a **supervised learning algorithm** used for both **classification** and **regression**. It is non-parametric, meaning it doesn't make any assumptions about the underlying data distribution, which makes it versatile for various applications. KNN works by analyzing the proximity or "closeness" of data points based on specific distance metrics.

```

def knn(query_point, dataset, k):

    distances = []

    for point in dataset:

        distance = compute_distance(query_point, point)

        distances.append((point, distance))

    distances.sort(key=lambda x: x[1]) # Sort based on distance

    neighbors = distances[:k]

    prediction = majority_vote(neighbors) # or average for regression

    return prediction

```

```

# Initialize the KNN classifier with k=3
knn = KNeighborsClassifier(n_neighbors=3)
# Train the model
knn.fit(X_train, y_train)
# Make predictions
y_pred = knn.predict(X_test)
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")
# Confusion matrix and classification report
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

```